

Streaming Communication for Scalable Data Exchange in DYNAMOS

Ruben Stoop
Complex Cyber-Infrastructure (CCI)
University of Amsterdam
Amsterdam, the Netherlands

Abstract—This thesis investigates how streaming communication can be integrated into DYNAMOS, a policy-driven middleware for dynamically composed microservice workflows. The original DYNAMOS data path is built around unary request-response transfers, which creates a bottleneck when large result sets must be moved through generated service chains. The thesis first compares candidate communication mechanisms in a controlled Kubernetes pipeline, using throughput, latency, pacing, correctness, recovery, and resource pressure as evaluation dimensions. It then implements three chunk-aware DYNAMOS data paths: chunked unary transfer, intra-job gRPC streaming, and RabbitMQ Streams at the agent boundary. A local validation on bulk `dataThroughTtp` retrieval shows that all three chunked implementations reduce completion time and peak memory compared with the original classic-unary baseline, with additional checks on `computeToData` and multi-provider `dataThroughTtp`. The results show that streaming is not one design choice but a set of architectural tradeoffs between raw transfer efficiency, temporal decoupling, recovery behaviour, and fit with DYNAMOS’ policy-driven execution model.

Index Terms—streaming communication, microservices, gRPC, event streaming, policy-driven middleware, DYNAMOS

I. INTRODUCTION

Many contemporary distributed systems operate in environments where data and computational resources are owned by different organizations. In such settings, data exchange must adhere to legal agreements, privacy regulations, and organizational policies. These constraints are common in domains such as healthcare, scientific research, and public-sector infrastructures, where unrestricted data exchange is not acceptable.

DYNAMOS (Dynamically Adaptive Microservice-based Operating System) is a data exchange middleware developed by the Complex Cyber-Infrastructure (CCI) group at the University of Amsterdam. The system implements a set of recurring data sharing archetypes that describe how data and computation are distributed among participants under policy constraints. DYNAMOS realizes these archetypes by dynamically composing chains of microservices that execute data sharing requests in compliance with formally evaluated policies [1], [2]. The middleware is designed to be adaptive, allowing architectural and execution decisions to change at runtime in response to policy requirements.

The current implementation of DYNAMOS uses gRPC for inter-service communication and relies primarily on atomic request-response interactions [1]. This communication model

is suitable for small or bounded data transfers but becomes limiting when applied to large-scale or continuous data exchange. In practice, data in distributed systems is often produced and consumed incrementally, for example in sensor data streams, distributed analytics pipelines, or staged processing of large research datasets.

When request-response communication is used in such scenarios, systems must either buffer large volumes of data before transmission or split transfers into multiple independent requests. Buffering increases memory usage and can lead to idle time between producers and consumers. Repeated request-response interactions introduce additional coordination overhead. Both effects negatively impact scalability and resource utilization as data volumes and the number of participating services increase.

Streaming-based communication has been proposed as a solution to these limitations. Streaming allows data to be transferred and processed incrementally, enabling overlap between computation and communication and reducing peak memory usage. gRPC supports several streaming interaction patterns, including client-side, server-side, and bidirectional streaming.

Integrating streaming communication into DYNAMOS is not straightforward. Streaming relies on long-lived connections and continuous data flows, which must be reconciled with DYNAMOS’ policy-driven execution model, dynamic microservice composition, orchestration decisions, and fault handling mechanisms. Without careful design, the introduction of streaming may interfere with policy enforcement or reduce the ability to evaluate and reason about system behavior.

The goal of this thesis is to investigate how streaming communication can be incorporated into DYNAMOS in a way that improves scalability and performance while preserving policy compliance and adaptive behavior. This involves analyzing existing streaming communication mechanisms, designing architectural extensions to the middleware, implementing streaming support, and evaluating the resulting system under realistic workloads.

The main research question addressed in this thesis is:

Main RQ: How can streaming communication be integrated into a dynamically adaptive, policy-driven microservice-based data exchange middleware such as DYNAMOS, while preserving compliance guarantees and improving scalability and performance?

To answer this question, the following sub-research questions are defined:

- **RQ1:** Which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS?
- **RQ2:** How can streaming communication be integrated into a microservice-based architecture in a way that preserves dynamic service composition, policy-driven orchestration, and isolation guarantees?
- **RQ3:** What is the impact of the implemented streaming data paths on scalability, performance, and robustness compared to a traditional request-response approach under realistic DYNAMOS workloads?

II. RELATED WORK

This section reviews existing literature related to streaming communication in microservice-based systems, with a particular focus on topics that are central to this thesis: streaming communication technologies and interaction models, architectural integration in adaptive microservice middleware, policy enforcement under continuous data exchange, resilience mechanisms for streaming workflows, and validation practices with respect to scalability and performance.

A. Streaming communication technologies and interaction models

DYNAMOS is implemented as a dynamically composed microservice system deployed on Kubernetes, using gRPC for inter-service communication and RabbitMQ for asynchronous messaging [1], [2]. This constrains the design space for introducing streaming functionality to a limited set of technologies that are compatible with containerized microservices and cloud-native deployment models. The primary question addressed in this body of work is which streaming communication mechanisms are suitable for large-scale or continuous data exchange, and under which conditions direct streaming RPC should be preferred over broker-based approaches.

gRPC provides native support for streaming through client-streaming, server-streaming, and bidirectional streaming RPCs. These interaction patterns differ in message flow control, lifetime of connections, buffering behaviour, and error propagation semantics. In the context of DYNAMOS, the choice of streaming pattern is not merely an API concern but has architectural consequences, as it affects memory usage, cancellation behaviour, and the ability to overlap communication and computation. The original DYNAMOS thesis identifies client-streaming as a promising mechanism for transferring large datasets incrementally, reducing peak memory pressure at the cost of increased handling complexity [1].

However, the use of long-lived streaming connections introduces operational challenges that are largely absent in unary request-response interactions. Starck and Ghofrani show that gRPC streaming complicates load balancing in Kubernetes environments, as persistent connections can cause traffic to

become unevenly distributed across replicas [3]. This observation is particularly relevant for systems such as DYNAMOS that rely on horizontal scaling and ephemeral execution units. Complementary benchmarking work demonstrates that the performance characteristics of streaming gRPC are sensitive to runtime configuration and virtualization overhead, with measurable effects on latency and resource utilisation in containerized environments [4].

An alternative to direct service-to-service streaming is to externalise the data plane to a messaging or event streaming platform. Broker-based communication decouples producers and consumers in time, delegates buffering and persistence to the broker, and can simplify fan-out scenarios. Comparative benchmarks of Kafka, RabbitMQ, NATS, and ActiveMQ Artemis show that different brokers dominate under different workload characteristics, reinforcing that broker choice is highly workload dependent [5]. A cloud-native literature review comparing Kafka, Pulsar, and RabbitMQ further highlights trade-offs between throughput, reliability, operational complexity, and Kubernetes integration [6]. While these studies are not conducted in the context of policy-driven middleware, they provide structured criteria that are directly applicable when evaluating broker-based alternatives for DYNAMOS.

Several studies also emphasize that streaming design is influenced not only by technology choice but by the underlying communication model. DataXc contrasts push-based streaming approaches with pull-based designs and reports improved latency stability and throughput under consumer-controlled flow [7]. Although the application domain differs, the distinction between producer-driven and consumer-driven streaming is directly relevant to backpressure management and resource utilisation in DYNAMOS-compatible workflows.

Overall, existing literature provides a broad comparison of streaming technologies and interaction patterns, but these comparisons are typically conducted in generic microservice or stream-processing settings. Existing studies are still useful methodologically because they emphasize identical workloads, common measurement points, latency distribution, throughput, resource pressure, and recovery behaviour as recurring benchmark dimensions [8]–[11]. The gap addressed in this thesis is therefore not the invention of a new benchmarking methodology, but the application of those dimensions to dynamically composed, policy-driven middleware with ephemeral execution units.

B. Architectural integration of streaming in adaptive microservice middleware

Beyond the selection of a streaming mechanism, integrating streaming communication into an adaptive microservice architecture raises architectural questions related to modularity, orchestration, and lifecycle management. DYNAMOS is explicitly designed around dynamic microservice composition, where execution chains are generated and deployed per request based on policy evaluation and system state [1], [2]. These chains are short-lived by design, which improves isolation

and compliance guarantees but complicates the management of long-lived communication channels.

Communication in DYNAMOS is abstracted through a sidecar-based architecture, decoupling communication concerns from core microservice logic. The sidecar pattern is widely used in cloud-native systems to address cross-cutting concerns such as observability, security, and traffic management. Figueira and Coutinho demonstrate how sidecars can be integrated into self-adaptive microservice architectures on Kubernetes, enabling runtime adaptation without violating service modularity [12]. While their work does not explicitly address streaming data transfer, it illustrates how communication behaviour can be adapted dynamically at runtime.

Several middleware frameworks operationalise sidecars as first-class runtime components. Figueira and Coutinho discuss sidecar-based runtimes such as Dapr, which provide a protocol-agnostic communication abstraction supporting service invocation, pub/sub, and state management through a sidecar process [12]. Such runtimes demonstrate that heterogeneous communication paradigms, including streaming and messaging, can be unified behind a stable interface. However, this class of runtimes assumes relatively static service topologies and does not address dynamic chain composition driven by policy evaluation, which limits their applicability as a direct solution for DYNAMOS.

Existing architectural work therefore supports the feasibility of integrating streaming into sidecar-based microservice systems but does not address the combination of long-lived streaming connections with ephemeral, dynamically composed execution chains. This limitation is particularly relevant in systems that combine adaptivity with strict policy enforcement requirements.

C. Policy enforcement and compliance under continuous data exchange

Policy enforcement is a defining characteristic of DYNAMOS, where access rights, execution locations, and data-flow constraints are evaluated prior to execution and enforced through dynamically composed microservice chains [1], [2]. In the current architecture, policy enforcement is implicitly scoped to bounded request-response interactions. Introducing streaming communication challenges this assumption, as data transfer may span longer periods during which policies, system state, or execution context may change.

Neumaier et al. propose a policy-aware architecture for decentralized dataset exchange that emphasizes continuous monitoring and runtime validation against machine-readable policies [13]. While their work does not explicitly target streaming RPC, it introduces an important distinction between one-time policy clearance and ongoing compliance monitoring, which becomes critical when data is exchanged incrementally over long-lived connections.

At the communication layer, several studies propose enforcing security and access control through proxies or sidecars. Thiyagarajan et al. present a proxy-based approach for securing gRPC communication using mTLS, JWT authentication,

and RBAC [14]. Although the focus is on security rather than data usage policy, the architectural approach is relevant, as it suggests that enforcement can be applied at communication boundaries and potentially at message granularity.

Despite these contributions, there is limited guidance on how policy enforcement should interact with long-lived streaming connections in systems that dynamically compose execution chains. In particular, existing work does not address how policy violations should be detected or handled mid-stream, or how streams should be terminated or adapted when compliance conditions change. These limitations highlight open challenges when introducing streaming into policy-driven middleware such as DYNAMOS.

D. Resilience, backpressure, and fault handling in streaming workflows

Streaming-based communication introduces operational challenges that are less pronounced in request-response systems. Long-lived data flows must tolerate partial failures, adapt to fluctuating load, and prevent uncontrolled resource growth through effective backpressure mechanisms. These challenges are compounded in DYNAMOS by dynamic microservice composition and ephemeral execution units.

Reactive microservice architectures emphasize asynchronous communication, non-blocking execution, and fault isolation as mechanisms for improving resilience under variable load [15]. Although this work does not focus on streaming RPC, its principles are directly applicable to streaming workflows, where failures and backpressure must be managed continuously rather than per request.

Studies on high-volume data processing in microservices highlight the importance of explicit flow control and decoupling between producers and consumers. Guntupalli and Vamshi show that event-driven architectures combined with messaging systems can mitigate bottlenecks and stabilize performance under sustained data rates [16]. While these systems typically rely on brokers rather than direct streaming RPC, they reinforce the need for explicit backpressure mechanisms in streaming data paths.

From a systems perspective, Gan and Delimitrou demonstrate that communication overhead and tail latency dominate the performance of large-scale microservice deployments [17]. Their findings underline that resilience and scalability issues often emerge from inter-service communication rather than computation, an effect that is amplified in streaming scenarios.

Existing literature provides well-established principles for resilience and fault tolerance in microservice systems, but these are typically studied either in request-oriented architectures or broker-based streaming platforms. There is limited work examining how these mechanisms interact with direct streaming RPC in dynamically composed microservice chains, leaving questions about how failures and backpressure should be handled in dynamically composed microservice systems.

E. Validation practices, scalability, and performance evaluation

Empirical evaluation of streaming communication often focuses on comparing streaming and non-streaming interaction styles in terms of throughput, latency, and resource usage. Studies focused specifically on gRPC streaming show that streaming RPCs outperform unary calls for large payloads and continuous data flows by enabling incremental processing and reducing buffering overhead [4], [18]. Complementary protocol-level comparisons between REST and gRPC further establish that binary, multiplexed transports outperform request-response HTTP under load, which provides additional context for choosing gRPC as the baseline communication layer [19], [20].

Scalability in containerized streaming systems has been studied through systematic mapping and empirical evaluations. A mapping study on performance analysis techniques for streaming workloads in containerized microservices identifies network contention, scheduling overhead, and coordination costs as primary scalability bottlenecks [10]. Empirical studies of streaming pipelines deployed on Kubernetes further show that streaming approaches scale more gracefully than request-response models when concurrency increases, provided that backpressure and flow control are properly configured [3].

However, most existing evaluations assume static microservice topologies and fixed execution paths. Prior performance studies rarely consider adaptive middleware that dynamically constructs execution chains based on policy evaluation. As a result, there is limited empirical insight into how streaming communication scales when embedded within dynamically adaptive, policy-driven systems such as DYNAMOS. This lack of combined evaluation of streaming, scalability, and adaptivity motivates the experimental focus of this thesis.

III. RQ1: TRANSPORT BENCHMARK DESIGN AND RESULTS

This section addresses *RQ1*: which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS? It combines a literature-backed comparison of candidate mechanisms with a controlled benchmark design that establishes a gRPC reference baseline before broker-based alternatives are added.

A. gRPC streaming

According to the official gRPC documentation, gRPC is based on service definitions in Protocol Buffers and supports four RPC styles: unary, server-streaming, client-streaming, and bidirectional streaming. For streaming calls, message ordering is preserved within each individual RPC, while deadlines, cancellation, metadata, and both synchronous and asynchronous APIs are supported by the runtime [21]. These properties make gRPC streaming the most direct extension of the current DYNAMOS communication model.

For microservice chains with predominantly point-to-point data transfer, client-streaming is the most natural first candidate because it lets a sender incrementally push chunks or records to a single downstream consumer. Bidirectional streaming becomes relevant when the receiver should provide progress feedback, pacing information, or explicit acknowledgements during transfer. The main advantages of gRPC streaming are low protocol overhead, direct reuse of protobuf contracts, and the absence of an additional broker layer. The disadvantages are that sender and receiver remain temporally coupled, durable replay is not provided by default, and backlog handling or crash recovery must be implemented at application level. Even so, prior work on asynchronous gRPC streaming demonstrates that carefully designed protocols can support efficient and even exactly-once style delivery patterns, which makes gRPC a strong experimental baseline rather than merely the status-quo option [4], [18], [19].

B. Apache Kafka

Apache Kafka is officially described as an open-source distributed event-streaming platform for high-performance pipelines, streaming analytics, data integration, and mission-critical applications. Its published core capabilities emphasise high throughput, durable storage, high availability, built-in stream processing, and large-scale horizontal scalability [22]. In architectural terms, Kafka exposes an append-only log abstraction in which messages are written to topics and partitions, allowing replay and independent consumption by multiple downstream readers [23].

Kafka is therefore the strongest candidate when temporal decoupling, replayability, or large consumer backlogs matter more than direct point-to-point simplicity. Its advantages are durable retention, repeated reads by independent consumers, and a mature ecosystem for analytics and downstream integration. The disadvantages are the extra operational footprint of a broker cluster, partition-local rather than global ordering, and a non-trivial tuning burden around acknowledgements, partitioning, batching, and replication. Recent review work also notes that Kafka benchmarks are frequently hard to reproduce because configuration details are underreported, which strengthens the case for a carefully controlled thesis benchmark [11], [23], [24].

C. RabbitMQ

Earlier comparative work positions RabbitMQ as a routing-oriented messaging system that offers flexible exchanges, acknowledgements, and strong support for enterprise-style asynchronous messaging [23], [25]. Current RabbitMQ documentation additionally introduces *streams* as an append-only, non-destructive, always persistent and replicated data structure that complements traditional queues rather than replacing them. Stream consumers can attach at arbitrary offsets or timestamps, replay data, and scale out via super streams when partitioning is needed [26].

This makes RabbitMQ attractive as a hybrid comparison point between classic broker semantics and newer log-like

stream semantics within the same ecosystem. Its advantages are flexible routing, explicit acknowledgements, and a gradual migration path from queue-oriented messaging to append-only streams. The limitations are that classic RabbitMQ semantics are not optimised for replay-heavy logs, while stream mode introduces its own replication and partitioning trade-offs. RabbitMQ should therefore be evaluated not only on raw throughput, but also on how well it balances routing flexibility, operational continuity, and stream-specific behaviour [23], [25], [26].

D. NATS with JetStream persistence

The NATS ecosystem deserves separate consideration because it targets lightweight, location-transparent distributed communication. Official NATS documentation presents core NATS as a connective technology for modern distributed systems with publish/subscribe and request/reply communication, while JetStream adds persistence, replay, replication, and consumer state on top of the base message bus [27]–[29]. JetStream explicitly supports replay from the beginning of a stream, from a specific sequence number, or from a time-based position, and it offers both push-based and pull-based consumers for different workload profiles [29].

For the thesis, NATS is interesting because it represents a lightweight alternative to Kafka and RabbitMQ rather than a direct substitute for either. JetStream supports durable streams, replay, replication, and scalable pull consumers, while core NATS preserves a simple request/reply and pub/sub model that is attractive for microservice systems [28], [29]. A key complication is that comparative literature still often discusses the older NATS Streaming layer rather than JetStream, for example Sharvari and Sowmya Nag [25], which means that published results do not always map perfectly to the current product. NATS with JetStream is most promising when lightweight deployment, elastic scaling, and a smaller operational surface are valued more highly than Kafka’s larger ecosystem and benchmarking history.

E. Benchmark Design

1) *Methodological positioning and scope:* The benchmark design adapts recurring evaluation principles to the specific constraints of DYNAMOS rather than copying a single prior protocol. The controlled multi-stage pipeline follows the style of prior streaming-pipeline evaluations such as DataXc [7]; identical business logic across mechanisms follows comparative benchmarking practice for distributed systems, where Karimov et al. require the same workload definitions and measurement points across systems under test, and Henning and Hasselbring define scalability as the rate of throughput growth per added resource unit [8], [9]; the emphasis on throughput, latency, concurrency, resource pressure, and recovery reflects recurring evaluation dimensions in streaming and IPC (Inter-process communication) studies [10], [11].

The aim is not to identify a universally best mechanism, but to narrow the design space for DYNAMOS by asking which communication model best fits large, ordered, and concurrent

inter-service transfer under realistic operational constraints. The completed corpus evaluates six transport mappings against the same pipeline, the same payload definitions, and the same correctness criteria, and the results are interpreted in terms of DYNAMOS integration. The sixth mapping is batched unary gRPC, in which one unary request carries multiple logical records. It is included as a first-class result because the later DYNAMOS implementation uses the same idea for chunked unary bulk transfer. The chapter body keeps the main comparison compact: the completed single-node clean matrix, three focused operational scenarios, and the resulting architectural interpretation. Full preset definitions and detailed per-case tables remain in the appendix.

2) *Common testbed and fairness constraints:* All candidate mechanisms were evaluated in the same Go-based microservice chain deployed on Kubernetes: *Producer* → *Transformer* → *Sink*. The services ran in a dedicated namespace using the same manifests, the same payload schema, and the same transformation logic. Resource budgets were controlled through Kustomize overlays so that the comparison remained centered on the communication layer rather than on application-level differences [8], [9].

The transport mappings were also kept as close as possible to one another. Client-streaming, per-message unary, and batched unary gRPC provide the direct service-to-service reference points. Kafka preserves per-flow ordering through keyed partitioning. RabbitMQ uses streams rather than classic queues. NATS uses JetStream with asynchronous publishing and explicit consumer acknowledgements. The brokered mappings were tuned for batched or asynchronous sending rather than for one blocking publish acknowledgement per record, so the comparison remains centered on transport behaviour rather than on intentionally slow acknowledgement patterns. Batched unary keeps the same logical messages as the other transports, but groups them into bounded unary requests so that the sink still measures logical records rather than RPC calls.

The service logic remained intentionally lightweight. The producer emitted synthetic or replayed records, the transformer applied a deterministic operation, and the sink recorded latency, pacing, and correctness summaries. That kept the benchmark focused on transfer behaviour, buffering, and recovery rather than on domain-specific computation [7], [10].

3) *Evaluation dimensions:* The literature consistently treats streaming evaluation as a multi-dimensional problem rather than a single throughput race [7], [10], [11]. This study follows that structure. Five dimensions matter most here: transfer efficiency, latency and pacing stability, overload behaviour, recovery correctness, and operational cost. A transport that leads the clean matrix but degrades badly under backpressure or restart is not automatically the best fit for DYNAMOS. The benchmark therefore keeps those dimensions visible throughout the chapter.

4) *Scenario set and scenario-to-property mapping:* The completed RQ1 study uses four scenario families: a clean bulk-transfer matrix, a steady-rate continuous scenario, a slow-consumer backpressure scenario, and a forced-restart

recovery scenario. This scenario set is not presented as a new benchmark methodology. It adapts recurring concerns from stream-processing and microservice evaluation literature, namely sustained transfer, latency stability, overload response, recovery after interruption, and correctness under at-least-once delivery [8], [10], [11], [30]. These properties are relevant to DYNAMOS because generated workflows may transfer large intermediate data, may contain stages with unequal processing rates, and may be restarted or redeployed as part of normal Kubernetes operation.

5) *Experimental factors*: The main independent variables are payload size, concurrency, target rate where applicable, sink slowdown in the backpressure case, and explicit failure injection in the recovery case. Concurrency denotes the number of parallel logical workers inside one benchmark run, not the number of producer Kubernetes jobs. In brokered modes, corresponding transformer and sink workers consume the worker-specific streams, subjects, or partitions. Resource budgets are held constant through Kubernetes overlays so that performance differences reflect the transport mapping rather than an uneven infrastructure allocation. This keeps the experiment matrix broad enough to expose real tradeoffs without turning the chapter into an exhaustive lab log [11], [25].

6) *Result presentation and appendix boundary*: The main text reports one clean-summary table, one compact size-wise clean breakdown, and three focused scenario tables for continuous flow, backpressure, and recovery. Detailed per-case clean-matrix tables remain in Appendix B. Resource metrics are used only where the sampling scope is the same across the compared cells. Broker-specific resource traces from exploratory runs are kept as supporting context rather than as a primary ranking signal. The RQ1 clean matrix should therefore be read as a screening study for candidate communication mechanisms; the DYNAMOS-in-situ evidence in RQ3 is the stronger evidence for the implemented chunked paths.

F. Results

1) *Metric definitions*: The benchmark reports the following metrics. The clean, continuous, and backpressure tables use the first group of metrics. The recovery table adds a second group because restart behaviour cannot be described by one number.

- **Message throughput**. Message throughput reports how many messages reach the sink per second. It is the main clean-matrix metric because it shows how much protocol and broker overhead each transport adds for the same logical workload.
- **Data throughput**. Data throughput reports the same transfer rate in MiB/s. It is included because two transports can move a similar number of messages while carrying different effective byte volumes once framing, batching, and message size are considered.
- **Median latency**. Median latency is the typical end-to-end time between producer emission and sink observation. It helps separate transports that are usually fast from transports that only look good in aggregate throughput.

- **p_{95} and p_{99} latency**. These percentiles report tail latency. They matter because broker queues, batching, and consumer polling can keep average throughput high while delaying the slowest messages.
- **p_{95} inter-arrival**. Inter-arrival delay measures the spacing between consecutive messages at the sink. In the continuous and backpressure scenarios it shows whether the sink receives a regular flow or bursty delivery.
- **Jitter**. Jitter summarizes variation in inter-arrival timing. A low-jitter transport delivers messages in a more predictable rhythm, which is useful when downstream services depend on steady pacing.
- **Duplicates**. Duplicate counts show how often the sink observes the same logical message more than once. This is important for at-least-once systems because replay, retries, or commit timing can preserve delivery while still requiring idempotent downstream processing. Sink throughput is computed from unique logical messages; duplicate deliveries are counted separately and do not inflate the reported message rate.
- **Ordering violations**. Ordering violations count messages that arrive out of the expected per-flow sequence. This matters for DYNAMOS-style pipelines because some transformations assume that chunks from the same logical stream remain ordered.
- **Pre-failure throughput**. In the recovery scenario, pre-failure throughput confirms whether the transport was near the configured target before the restart. Without this value, a recovery result could be misleading because a transport might appear stable only because it was already under-producing.
- **Recovery-window throughput**. Recovery-window throughput measures delivery during the fixed window immediately after the forced restart. It shows how much useful progress continues while the system is reconnecting or replaying.
- **Post-recovery throughput**. Post-recovery throughput measures the rate after the immediate disruption has passed. It shows whether the transport returns to normal transfer behaviour or remains degraded after the restart.
- **First post-failure message**. This is the time until the first message is observed after the restart. It measures visible resumption, but not full recovery, since a transport can emit one message quickly while still carrying a large backlog.
- **Sustained 80%/5s recovery**. This is the first point at which the sink observes five complete one-second buckets at or above 80% of the target rate. It is stricter than first-message recovery because it requires stable progress rather than a single successful delivery.
- **Recovery debt**. Debt is the number of messages missing from the fixed recovery window relative to the configured target. It represents how much work the transport failed to deliver on time and therefore how much backlog or lost progress remains after disruption.
- **Recovery p_{95} latency**. This is the tail latency of messages

Table I
SCENARIO-TO-PROPERTY MAPPING USED IN THE RQ1 BENCHMARK.

Scenario	Primary property	Why it matters for DYNAMOS
Clean bulk transfer	Protocol overhead, sustained throughput, multi-hop transfer efficiency	Large dataset exchange should remain efficient when data moves incrementally through a composed service chain.
Continuous steady-rate flow	Latency stability, inter-arrival regularity, pacing precision	Long-lived flows matter when downstream services depend on a predictable delivery rhythm.
Slow-consumer backpressure	Queue build-up, overload propagation, throttling versus delay accumulation	DYNAMOS pipelines can contain stages with unequal service rates, so overload handling must remain interpretable.
Forced-restart recovery	Reconnect behaviour, duplicates, ordering preservation, recovery time	Ephemeral microservices and restarts are normal in Kubernetes-style deployments.

observed in the recovery window. It reveals whether recovery is happening through smooth resumed delivery or through delayed backlog bursts.

- **Post-failure duplicates and ordering violations.** These are correctness counts restricted to the part of the run after the restart. Separating them from clean-run correctness makes it visible whether failures trigger replay side effects.
- **Backlog drain and completion.** Backlog drain measures how long it takes for delayed work to clear after the restart. The completion flag records whether the run eventually reached the expected message count.
- **Resource pressure.** CPU and memory measurements are used as supporting evidence where the same sampling scope is available across the compared runs. They indicate whether a transport improves throughput by adding acceptable overhead or by shifting pressure to the broker layer.

2) *Comparison scope:* The primary comparison in this chapter is a single-node Kubernetes benchmark across six transport mappings: client-streaming gRPC, per-message unary gRPC, batched-unary gRPC, RabbitMQ Streams, NATS JetStream, and Kafka. Single-node means that the pipeline services and broker components are scheduled on one Kubernetes node. This isolates transport overhead without adding cross-node replication or placement effects. Kafka, RabbitMQ Streams, and NATS JetStream can also be deployed as multi-node broker clusters, where partitioning, replication, and placement can change both throughput and resilience [6], [22], [26], [29]. Exploratory clustered broker runs were performed, but they are used as supporting context rather than as primary evidence because they expand the experiment from DYNAMOS communication design into broker-cluster tuning.

All RQ1 tables in the main text report arithmetic means over five repetitions unless stated otherwise. This follows the preferred reporting style for the thesis. The text still discusses visible spread and non-monotonic cells where they affect interpretation, because Kubernetes and broker timing can introduce outliers that a mean alone would hide.

Table II shows two important results. First, client-streaming and batched unary form the fast direct-transfer band. Client-

streaming has the highest average message throughput, but batched unary is close and has lower average tail latency in this matrix. This matters for DYNAMOS because it shows that the decisive property is amortising per-message overhead, not streaming as a label. A request-response mapping becomes competitive when one request carries a bounded batch of records instead of one logical message. Second, per-message unary remains much slower. It has low latency for individual requests because it never builds deep queues, but it pays the full request-response cost for every message and is therefore a lower-bound mapping rather than a realistic bulk-transfer design.

The brokered transports form a separate band. NATS JetStream has the highest average brokered throughput, followed by RabbitMQ Streams and Kafka. The price is substantially higher tail latency, caused by broker persistence, routing, consumer polling, and backlog effects. Ordering violations are absent in the clean matrix. NATS and Kafka produce duplicate deliveries, especially NATS with 66,863 duplicates across the clean matrix. These duplicates do not inflate the throughput number because the sink counts unique logical messages for throughput and records duplicate observations separately. For NATS, the likely mechanism is explicit acknowledgement with a configured 30 s `AckWait`: if processing or acknowledgement falls behind that window, JetStream can redeliver a message.

3) *Clean bulk-transfer matrix:* The size-wise breakdown is reported in Appendix D, Table XIV. It supports the same conclusion as the main clean summary: client-streaming and batched unary remain close across the tested payload range, while per-message unary stays far behind for bulk transfer. Several brokered cells are non-monotonic, so the robust conclusion is the band-level ordering rather than the exact ranking of every broker at every payload and concurrency. The direct gRPC mappings dominate local throughput, with batched unary close enough to client-streaming to justify taking a chunked-unary design into DYNAMOS. The brokered mappings are slower locally, but they still matter architecturally because they provide decoupling and replay semantics that direct streams do not provide by themselves.

Table II

SINGLE-NODE SYNTHETIC-CLEAN SUMMARY ACROSS SIX TRANSPORTS, AVERAGED OVER FIVE REPETITIONS, FOUR PAYLOAD SIZES, AND FOUR CONCURRENCY LEVELS.

Transport mode	Avg. msg/s	Avg. MiB/s	Avg. p_{95}	Avg. p_{99}	Duplicates	Ordering viol.
Client-streaming	52,990.79	103.46	108.68	130.91	0	0
Batched unary	51,384.68	104.18	51.63	67.25	0	0
Unary	3,079.83	12.77	1.39	24.82	0	0
RabbitMQ Streams	13,900.44	31.27	906.37	944.03	0	0
NATS JetStream	14,525.50	36.98	1,518.21	1,607.89	66,863	0
Kafka	11,218.10	27.20	1,465.02	1,599.80	65	0

Table III

CONTINUOUS STEADY-RATE SCENARIO: SINGLE-NODE RESULTS AVERAGED OVER FIVE REPETITIONS (1 KiB PAYLOAD, 1,000 MSG/S TARGET). THROUGHPUT IN MSG/S; LATENCY, INTER-ARRIVAL, AND JITTER VALUES IN MILLISECONDS.

Transport mode	Concur.	Throughput	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	4	900.62	5.87	1.53	0.64
Client-streaming	8	904.19	4.49	1.63	0.69
Batched unary	4	1,050.37	376.29	0.00	13.85
Batched unary	8	901.17	766.72	0.00	20.22
Unary	4	901.04	0.71	2.00	1.10
Unary	8	897.49	1.11	2.07	1.01
RabbitMQ Streams	4	895.16	29.84	2.80	1.31
RabbitMQ Streams	8	1,044.28	34.08	2.64	1.89
NATS JetStream	4	903.02	15.71	1.78	0.75
NATS JetStream	8	908.53	24.86	1.78	0.72
Kafka	4	904.97	356.40	0.01	8.20
Kafka	8	891.07	379.80	0.02	8.71

Table IV

BACKPRESSURE SCENARIO: SINGLE-NODE RESULTS AVERAGED OVER FIVE REPETITIONS (1 KiB, CONCURRENCY 8, 4,000 MSG/S TARGET, 2 MS SINK DELAY). THROUGHPUT IN MSG/S; LATENCY, INTER-ARRIVAL, AND JITTER VALUES IN MILLISECONDS.

Transport mode	Throughput	Median lat.	Avg. p_{95}	p_{95} inter-arrival	Jitter
Client-streaming	3,353.33	1,536.17	1,919.99	2.14	0.71
Batched unary	3,486.84	114.07	215.16	1.15	0.54
Unary	2,316.50	0.44	0.96	3.10	1.04
RabbitMQ Streams	3,057.96	1,769.03	2,479.59	2.15	0.70
NATS JetStream	3,234.54	1,569.85	2,641.20	1.97	0.61
Kafka	1,338.26	2,499.33	10,507.86	2.24	0.94

4) *Continuous steady-rate scenario*: The continuous scenario uses 1 KiB messages and a target rate of 1,000 msg/s. The purpose is not to maximize throughput, but to see how steadily each transport sustains an ongoing flow.

All transports remain near the configured target, so the comparison is mainly about pacing. Unary has the lowest tail latency because each message is acknowledged independently and the producer naturally slows down. Client-streaming also has low tail latency and regular inter-arrival times. Batched unary sustains the target, but its high p_{95} latency and near-zero inter-arrival percentile show bursty delivery: records arrive in batches rather than as a smooth stream. That is acceptable for bulk transfer, but less attractive for workloads that need steady per-message pacing. Among the brokered transports, NATS and RabbitMQ Streams have moderate tails, while Kafka remains burst-dominated with high jitter.

5) *Backpressure scenario*: The backpressure scenario keeps the payload at 1 KiB and concurrency at 8, but adds a 2 ms sink delay while the producer targets 4,000 msg/s. This exposes how each transport behaves once downstream processing becomes the bottleneck.

Batched unary is the strongest backpressure result in this run. It achieves the highest realised throughput and keeps

the tail much lower than client-streaming and the brokered mappings. Client-streaming, NATS, and RabbitMQ Streams all keep useful throughput under pressure, but they absorb the overload as queueing delay, with median latencies around 1.5–1.8 s and p_{95} latencies around 1.9–2.6 s. Unary gives up throughput earlier and therefore keeps the requests that complete very low-latency. Kafka is the weakest backpressure result, with the lowest throughput and a p_{95} latency above 10 s. It also produces 83 duplicate observations across the five repetitions, again without inflating unique-message throughput.

6) *Recovery scenario*: The recovery scenario uses 4 KiB messages at 2,000 msg/s and forces a transformer restart after 3 seconds. The table separates the run into pre-failure throughput, a fixed 10 s recovery window, and the post-recovery tail so that restart visibility, sustained recovery, backlog drain, and correctness cost are not collapsed into one ambiguous number.

Before failure, all transports are close enough to the configured target to make the recovery comparison meaningful. Client-streaming produces the fastest visible resumption, but its current reconnect policy replays the worker stream and therefore creates 10,505 post-failure duplicates on average. Its recovery-window throughput is also low, so the fast first message does not mean the stream has fully recovered. Batched unary and RabbitMQ Streams have a better balance between recovery-window throughput and debt. Batched unary has no duplicate or ordering cost, while RabbitMQ Streams resumes earlier but produces 929 post-failure duplicates and much higher recovery-window tail latency.

Table V

RECOVERY SCENARIO: SINGLE-NODE RESULTS AVERAGED OVER FIVE REPETITIONS (4 KiB, CONCURRENCY 8, 2,000 MSG/S, TRANSFORMER RESTART AT 3 s). THROUGHPUT VALUES ARE IN MSG/S; TIME, LATENCY, AND DRAIN VALUES ARE IN MILLISECONDS. SUSTAINED RECOVERY IS THE FIRST FIVE COMPLETE ONE-SECOND BUCKETS AT OR ABOVE 80% OF TARGET. DASHES INDICATE THAT THE VALUE WAS NOT OBSERVED.

Transport mode	Pre msg/s	Recovery msg/s	Post msg/s	First post-fail	Sustained 80%/5s	Debt	Recovery p_{95}	Post-fail dup.	Post-fail order	Backlog drain	Done
Client-streaming	2,685	348	1,994	0.89	10,000	16,516	0.62	10,505	0	29,439.54	Yes
Batched unary	2,075	1,810	1,626	2,177.25	2,000	4,500	396.22	0	0	25,909.19	Yes
Unary	2,215	1,464	1,804	2,112.04	12,000	7,267	2.08	0	0	26,025.59	Yes
RabbitMQ Streams	2,105	1,657	2,041	298.33	2,000	3,575	2,594.73	929	0	25,371.16	Yes
NATS JetStream	2,297	2,124	1,092	103.93	0	20	288.66	0	8	30,245.95	Yes
Kafka	2,356	0	1,822	32,513.54	–	20,000	–	131	0	33,991.45	Yes

NATS JetStream has the strongest recovery-window throughput in this five-repetition run and almost no throughput debt, but it records eight post-failure ordering violations on average and drains later than the other non-Kafka transports. Kafka is the clearest negative case: no messages arrive in the fixed recovery window, first visible recovery is delayed by more than 32 s, and post-failure duplicates still occur. The recovery table therefore does not select the same winner as the clean matrix. It shows a tradeoff between fast resumption, replay cost, ordering, and backlog drain.

7) *Discussion*: RQ1 does not point to one transport that should be used everywhere in DYNAMOS. It identifies which mechanisms are promising at which boundary. For local service-to-service transfer, direct gRPC mappings are strongest. Client-streaming gives the highest raw clean throughput, but batched unary is close enough to change the interpretation of the unary result. The weak unary column is not a verdict against request-response communication. It is a verdict against one-request-per-message transfer. Once multiple records are carried per request, unary becomes a strong bulk-transfer mechanism.

This is why RQ2 and RQ3 focus on chunk preservation rather than on replacing every boundary with a stream. DYNAMOS has two communication regimes. Inside generated jobs, services already communicate over gRPC, so direct streaming can be used where the sender and receiver lifetimes are naturally coupled. Across agents, DYNAMOS relies on sidecar-mediated RabbitMQ communication, dynamic job readiness, and retry logic. At that boundary, a chunked message model fits the existing architecture better than a single long-lived direct stream.

The brokered transports remain useful design evidence even though they are slower in the local clean matrix. RabbitMQ Streams is not validated here as a local latency improvement over chunked unary. Its value is architectural: retained records, offset-based replay, and decoupled consumption may matter more in a distributed DYNAMOS deployment than on a single node. NATS and Kafka are informative comparison points, but their clean-run duplicate behaviour, recovery behaviour, and larger platform shift make them lower-priority choices for the current DYNAMOS implementation.

The resulting RQ1 answer is boundary-specific. The best raw transport is client-streaming gRPC. The best request-shaped bulk-transfer pattern is batched unary. The best current DYNAMOS implementation hypothesis is therefore chunked transfer at existing boundaries, with gRPC streaming as an in-pod optimisation and RabbitMQ Streams as a distributed-boundary candidate rather than as the source of the local speedup.

G. Threats to Validity

Four limits are important when interpreting these results. First, all primary RQ1 evidence comes from one local single-node Kubernetes environment. It is suitable for isolating transport overhead, but it does not capture cross-node latency, broker placement, replication, or geographically distributed deployment. Second, the benchmark uses synthetic payloads and fixed request shapes. This makes the transports comparable, but it also means that the RQ1 results should be read as mechanism-level evidence rather than as a complete DYNAMOS workload evaluation. The integrated DYNAMOS benchmarks in RQ3 therefore carry more weight for the final implementation choice. Third, the tables report arithmetic means over five repetitions. This follows the thesis reporting preference, but Kubernetes scheduling, broker flushing, and container resource sharing can still introduce run-to-run variation. Small differences between brokered transports should therefore not be treated as decisive. Fourth, resource measurements are supporting evidence rather than the basis for the transport ranking. They are reported only where the same sampling scope is available across compared runs, because otherwise a broker-heavy transport could appear more expensive simply because more of its infrastructure was included in the measurement.

IV. STREAMING INTEGRATION IN DYNAMOS

This section addresses RQ2 by moving from a generic transport benchmark to an implementation inside DYNAMOS. The question is no longer only which communication mechanism is fast. The implementation must also preserve DYNAMOS' policy approval path, generated job model, sidecar-mediated communication, and isolation between data stewards.

A. Motivation

The original DYNAMOS implementation already identifies large message transfer as a limitation. The DYNAMOS thesis notes that the default maximum size of a gRPC method is around 4 MB, corresponding to approximately 30,000 records in the original experiments, and that the current gRPC implementation uses unary RPCs: one request and one response [1]. The same discussion observes that increasing the message size is theoretically possible but memory-heavy, and explicitly names streaming as the unfinished solution. This makes large-result transfer an appropriate point at which to evaluate streaming techniques inside DYNAMOS rather than only in a synthetic pipeline.

The implementation target is deliberately different from the average-query example commonly used in DYNAMOS demonstrations. An average query is a reduction workload: it may scan many records but returns one scalar result. That tests computation near the data source, but it does not test whether DYNAMOS can transport many aligned rows. Bulk retrieval therefore stresses the part of DYNAMOS that is most affected by full-response buffering. It is also a useful precursor to later distributed learning scenarios, but this thesis first validates whether the core DYNAMOS data path can move large row sets incrementally.

B. DYNAMOS execution path

DYNAMOS separates the control path from the data path. A client request first passes through the gateway, policy evaluation, orchestration, and the relevant agents. Only after that approval and composition step are generated job pods used to execute the request. For the `dataThroughTtp` archetype, the data provider job retrieves rows, the Trusted Third Party (TTP) or SURF-side job performs the compute-provider part of the chain, and the response eventually returns through the agent and gateway path.

Figure 1 shows the boundary that matters for the implementation. In the original path, a large result can be rebuilt several times before the client sees anything: inside the SQL response, inside the generated service chain, inside the agent response, and again at the gateway. The streaming implementation therefore treats chunk preservation as the central requirement. A transport is only useful for this workload if the first rows can leave the provider path before the final rows have been read and serialized.

The implementation should not be a special case for one benchmark request. DYNAMOS supports multiple archetypes and multiple data providers, so a chunk-aware path must keep existing request shapes valid. The primary evaluation

uses `dataThroughTtp` with one provider to isolate the large-result bottleneck. RQ3 then adds focused checks for `computeToData` and for `dataThroughTtp` with two providers, because those scenarios exercise different parts of the generated job and aggregation path.

C. Common chunk model

All chunked variants use the same logical stream model. The SQL query service reads the bulk result in fixed row batches. Each batch carries stream metadata: provider identity, correlation identifier, sequence number, row count, partial or final status, and ordered column names. The ordered-column metadata is important because Go protobuf structs can otherwise expose map fields in nondeterministic order, which would make equal results appear different across transports.

The aggregate and algorithm stages preserve chunk boundaries for non-average bulk workloads. They do not merge all incoming rows into one table unless the request explicitly uses the classic unary baseline. The gateway exposes chunk progress to the benchmark client through NDJSON `providerResult` events, with partial events before the final event. This keeps the external response shape compatible with the existing gateway while making progress visible.

Policy enforcement is not moved into the stream. DYNAMOS still evaluates the request before execution, constructs the permitted job path, and then lets that generated path process the approved data exchange. The current implementation therefore preserves the original policy decision point. It does not claim per-chunk policy revocation during a running stream.

D. Implemented variants

Four paths are compared in the thesis. The first is the original DYNAMOS style and the other three are chunk-aware implementations. This distinction matters because the RabbitMQ Streams path still needs chunking before it reaches the broker. If the generated job first built one giant result and only then wrote it into RabbitMQ Streams, the client would not receive an earlier first result and the memory problem would largely remain.

The classic unary baseline uses the original DYNAMOS main branch at commit `a2d9531` [31]. For the benchmark, its gRPC and message-size limits are configured high enough to complete the tested row counts. This avoids treating the historical 4 MB ceiling as the result. Classic unary remains intentionally full-buffered, because that is the behaviour the streaming implementations are meant to improve. The multi-provider classic baseline also includes a minimal aggregation correction so the first provider is not appended twice during result merging. This correction makes the baseline result valid; it does not add streaming behaviour.

The three chunked paths use the streaming implementation branch at commit `923b018` [32]. Chunked unary is the least disruptive path. It keeps unary calls and the existing agent style, but it sends many smaller result messages instead of one large result. Intra-job gRPC streaming replaces repeated unary calls inside the generated service chain with one stream. That

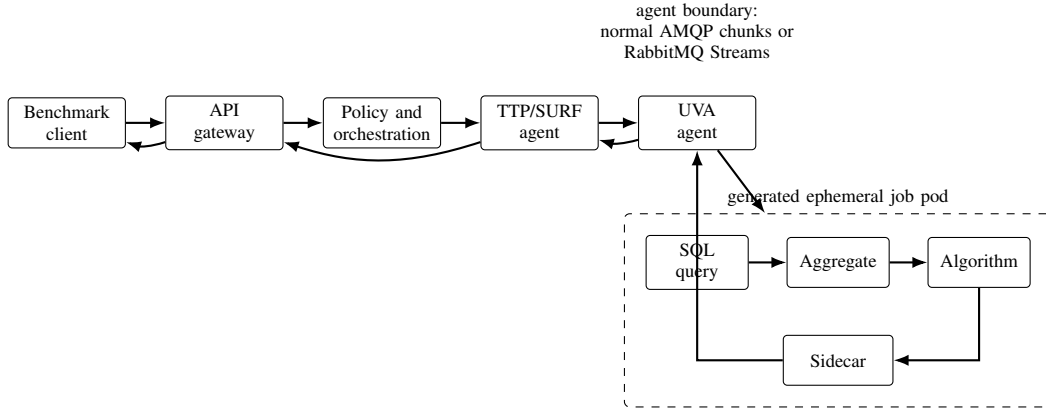


Figure 1. DYNAMOS bulk retrieval path used for RQ2. The implementation changes the bulk data path after policy approval. Chunking starts at the SQL query service and must be preserved through the generated job, agent path, and gateway response to avoid rebuilding one complete result table before the client sees progress.

Table VI
DYNAMOS IMPLEMENTATION VARIANTS AND THE ARCHITECTURAL BOUNDARY CHANGED BY EACH VARIANT.

Variant	In-pod service chain	Agent boundary	Buffering behaviour	Architectural purpose
Classic unary	Unary gRPC, full response	Classic RabbitMQ message path	Complete result is buffered into one logical response	Baseline closest to original DYNAMOS once the gRPC size ceiling is raised.
Chunked unary	Repeated unary calls carrying row chunks	Classic RabbitMQ carries separate chunk messages	Chunks stay separate through aggregate, algorithm, agent, and gateway	Smallest architectural change; preserves the message-driven style.
Intra-job gRPC streaming	Long-lived <code>SendDataStream</code> through the generated service chain	Classic RabbitMQ carries separate chunk messages	Chunks stream inside the pod and remain chunks after leaving it	Tests native gRPC streaming where DYNAMOS already uses local gRPC.
RabbitMQ Streams at agent boundary	Chunked unary path inside the generated job	RabbitMQ stream records replace classic queue messages for chunk transfer	Local chunks are written as broker stream records and reassembled downstream	Tests stream-log semantics for distributed inter-agent transfer.

matches gRPC’s native streaming model [21], but it couples sender and receiver lifetimes more tightly. RabbitMQ Streams changes the agent boundary. It keeps the chunked local path, then uses RabbitMQ’s append-only stream abstraction for chunk transfer between agents [26].

These variants are not three independent full-stack streaming systems. They are interventions at different parts of the same DYNAMOS path. The local performance improvement of the RabbitMQ Streams variant is therefore not caused by RabbitMQ Streams alone. It is caused by the common chunk-aware data path, with RabbitMQ Streams adding a different broker representation at the agent boundary.

E. RQ2 answer

Streaming can be integrated into DYNAMOS by changing the bulk data path after policy approval while leaving the control path intact. The implementation introduces an explicit chunk model at the SQL query service, preserves those chunks through aggregate and algorithm services, carries them through the agent boundary, and exposes partial result events to the client. This keeps dynamic service composition and policy-driven orchestration in place, while removing the need to repeatedly materialize one complete result table before progress is visible. Compatibility with `computeToData`

and multi-provider `dataThroughTtp` is treated as part of the implementation requirement rather than as an optional benchmark extension.

The best architectural choice is boundary-specific. Chunked unary is the best current local default because it gives chunking without changing the DYNAMOS execution model very much. Intra-job gRPC streaming is useful where both endpoints are already in the generated pod and a direct stream is acceptable. RabbitMQ Streams is more relevant at the inter-agent boundary, especially for future distributed deployments where retained streams, replay, and independent consumers may matter more than local latency.

V. EVALUATION OF DYNAMOS STREAMING IMPLEMENTATIONS

This section addresses RQ3 by validating the implemented DYNAMOS data paths on a large-result workload. RQ2 explains how the variants are integrated. RQ3 measures whether those variants improve completion time, time-to-first-result, and resource usage compared with the classic unary baseline.

A. Benchmark method

The primary benchmark uses the `dataThroughTtp` archetype with a single data provider, UVA. The request still

follows the TTP/SURF compute-provider path defined by the archetype, but the data fanout is intentionally limited to one provider so that the experiment isolates large-result transfer rather than multi-provider coordination.

Each request executes a bulk projection query over the large `PersonenLarge` and `AanstellingenLarge` tables and joins them on `Unieknr`. The result is limited to 50,000, 100,000, or 250,000 rows. The chunked variants use 5,000 rows per SQL chunk, which gives 10, 20, and 50 total chunks respectively. This value is an experimental configuration rather than a claimed optimum: it is large enough to avoid turning the benchmark into a per-row messaging test and small enough to expose partial progress. In the saved 50k-row bulk output, one 5,000-row chunk serialized as a DYNAMOS protobuf `MicroserviceCommunication` is approximately 280,255 bytes, or 273.7 KiB. That is larger than the final RQ1 synthetic payload range, so RQ3 is treated as the authoritative evidence for realistic DYNAMOS chunk sizes.

Four implementations are compared. Classic unary is taken from the original DYNAMOS main branch at commit `a2d9531`. The chunked unary, intra-job gRPC streaming, and RabbitMQ Streams variants are taken from the streaming implementation branch at commit `923b018`. Every primary `dataThroughTtp` cell uses 10 warm repetitions. Runs are accepted only when row counts match the expected limit and canonical content hashes validate the returned content. For chunked variants, the benchmark also requires partial result events before completion.

Two focused scenario checks are added after the primary benchmark. The first changes the archetype to `computeToData`. The second keeps `dataThroughTtp` but uses two data providers, `UVA`, `VU`. These checks test whether chunk preservation remains compatible when the generated job topology and aggregation path differ from the primary single-provider TTP case.

Resource usage is sampled from Kubernetes once per second across the relevant DYNAMOS namespaces and summarized as peak total CPU millicores and peak total memory in MiB. These sampled peaks are useful for comparing memory pressure, but they may underestimate short instantaneous spikes between samples. The measured request window starts immediately before the benchmark client submits the HTTP request and ends when the final response or final NDJSON event is observed. This means policy handling, orchestration, generated job creation, pod startup, data transfer, and response serialization are inside the measured window for both classic and chunked variants. The classic unary baseline required generated `surf`, `uva`, and `vu` jobs to be cleaned between repetitions. That cleanup happens before and after a measured request, not inside the request timing. Without it, later repetitions could return fast 502 or empty-provider responses because stale generated jobs were still present. The cleaned baseline is therefore the comparison used here.

Table VII
AVERAGE COMPLETION LATENCY FOR DYNAMOS BULK RETRIEVAL.
ALL CELLS COMPLETED 10/10 VALID REPETITIONS.

Implementation	50k rows	100k rows	250k rows
Classic unary	14.529s	22.791s	34.865s
Chunked unary	8.568s	7.869s	14.357s
Intra-job gRPC streaming	8.431s	11.074s	15.257s
RabbitMQ Streams boundary	7.647s	9.464s	17.302s

B. Completion latency

Table VII and Figure 2 show that all three chunk-aware variants reduce average completion latency compared with classic unary. At 250k rows, chunked unary is 2.43x faster than classic unary, intra-job gRPC streaming is 2.29x faster, and the RabbitMQ Streams boundary variant is 2.01x faster. The 100k row case is especially favourable to chunked unary, which finishes in 7.869s compared with 22.791s for classic unary. The relatively small increase of the chunked variants between 50k and 100k rows is explained by the fixed control-path cost analysed with Table IX: at these sizes much of the measured time is policy handling, orchestration, and job startup rather than transfer.

The average values include the long-tail runs that occur in the local Kubernetes deployment, so they are more conservative than medians. At 250k rows, classic unary ranged from 29.615s to 50.854s. Chunked unary ranged from 10.985s to 23.233s, intra-job gRPC streaming from 10.566s to 23.311s, and RabbitMQ Streams from 12.724s to 26.520s. This spread shows why repeated runs are needed, but the gap between classic unary and the chunked paths is large enough that it does not depend on one unusually fast or slow run.

C. Additional scenario checks

Table VIII shows that the chunk-aware paths are not limited to the primary single-provider TTP case. For `computeToData`, the improvement is smaller because the request avoids part of the TTP aggregation path, but the chunked variants still complete faster than classic unary at both tested sizes. For multi-provider `dataThroughTtp`, the improvement remains large: at 250k rows per provider, the classic baseline returns 500k total rows in 33.920s, while the chunked variants complete in 12.928s to 14.883s.

These scenario checks are compatibility-oriented rather than an exhaustive benchmark matrix. Their value is that they show the implementation still works when the archetype and provider count change, and that the speedup is strongest when DYNAMOS must move large row sets through the TTP path.

D. Time to first result

Table IX shows the qualitative benefit of chunking. Classic unary cannot expose progress: the first result is the final result. The chunked variants emit partial results substantially earlier. At 250k rows, the first partial result appears after about 6.5s to 8.0s on average, while completion follows after 14.4s

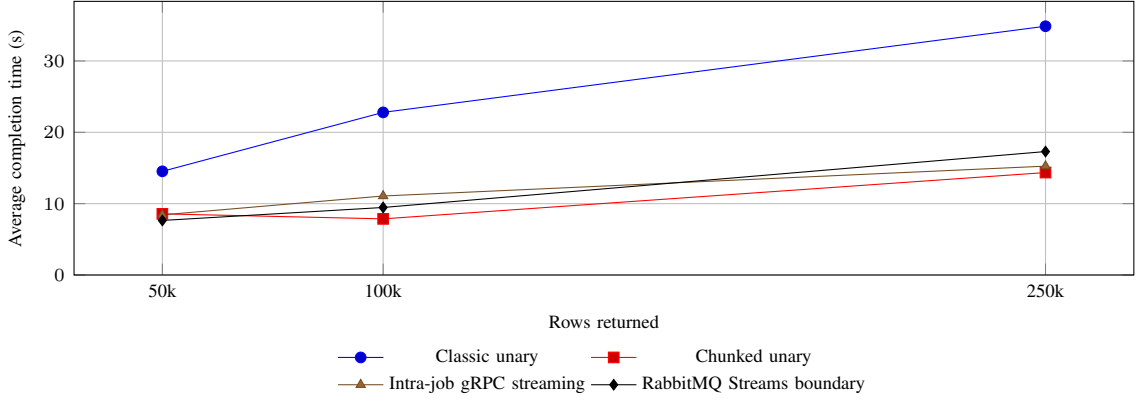


Figure 2. Average completion time for the local DYNAMOS bulk retrieval benchmark. Chunking is the dominant improvement: all chunk-aware variants scale better than classic unary as the returned row count grows.

Table VIII
FOCUSED DYNAMOS SCENARIO CHECKS BEYOND THE PRIMARY SINGLE-PROVIDER `DATAThroughTtp` BENCHMARK.

Scenario	Providers	Rows per provider	Total rows	Classic unary	Chunked unary	Intra-job gRPC streaming	RabbitMQ Streams boundary
computeToData	UVA, VU	50k	100k	9.660s	5.584s	4.895s	5.371s
computeToData	UVA, VU	250k	500k	16.290s	10.913s	10.876s	13.140s
dataThroughTtp	UVA, VU	50k	100k	13.163s	5.820s	5.322s	7.226s
dataThroughTtp	UVA, VU	250k	500k	33.920s	13.444s	12.928s	14.883s

Table IX
TIME-TO-FIRST-RESULT AND PARTIAL RESULT BEHAVIOUR. CLASSIC UNARY HAS NO PARTIAL RESULTS, SO FIRST RESULT EQUALS COMPLETION.

Implementation	Rows	First result	Done	Post-first	Partial chunks
Classic unary	50k	14.529s	14.529s	0.000s	0
Classic unary	100k	22.791s	22.791s	0.000s	0
Classic unary	250k	34.865s	34.865s	0.000s	0
Chunked unary	50k	7.712s	8.568s	0.856s	9
Chunked unary	100k	6.056s	7.869s	1.812s	19
Chunked unary	250k	6.492s	14.357s	7.866s	49
Intra-job gRPC streaming	50k	7.414s	8.431s	1.017s	9
Intra-job gRPC streaming	100k	8.816s	11.074s	2.259s	19
Intra-job gRPC streaming	250k	8.032s	15.257s	7.226s	49
RabbitMQ Streams boundary	50k	6.812s	7.647s	0.835s	9
RabbitMQ Streams boundary	100k	6.997s	9.464s	2.467s	19
RabbitMQ Streams boundary	250k	7.630s	17.302s	9.672s	49

to 17.3s. The post-first column estimates how much of the measured time remains after the first visible chunk. It is not a pure network-transfer measurement, because the first chunk still includes policy handling, job creation, and initial SQL execution. It is nevertheless useful because it separates request startup from the observable chunk-drain phase. Read together with Table VII, this decomposition shows that the end-to-end ratios understate the data-path difference: every chunked variant spends roughly 6–8 s before the first chunk, which is the shared policy, orchestration, and job-startup cost. End-to-end remains the headline metric since it is what a DYNAMOS client experiences, but the time-to-first-result values show that chunking also changes the interaction pattern: a client can observe progress and begin processing data before the full result has been materialized. This decomposition also matters

Table X
PEAK SAMPLED RESOURCE USAGE DURING DYNAMOS BULK RETRIEVAL. CPU IS PEAK TOTAL MILLICORES; MEMORY IS PEAK TOTAL MiB ACROSS SAMPLED KUBERNETES NAMESPACES.

Implementation	Rows	Peak CPU	Peak memory
Classic unary	50k	782m	1,571 MiB
Classic unary	100k	813m	2,046 MiB
Classic unary	250k	1,198m	3,695 MiB
Chunked unary	50k	403m	1,151 MiB
Chunked unary	100k	469m	1,723 MiB
Chunked unary	250k	1,097m	1,378 MiB
Intra-job gRPC streaming	50k	351m	1,198 MiB
Intra-job gRPC streaming	100k	479m	1,230 MiB
Intra-job gRPC streaming	250k	1,418m	1,495 MiB
RabbitMQ Streams boundary	50k	457m	1,161 MiB
RabbitMQ Streams boundary	100k	982m	1,436 MiB
RabbitMQ Streams boundary	250k	1,687m	1,715 MiB

for diagnosing whether a large request is still making progress.

E. Resource usage

The resource data in Table X support the main architectural claim. At 250k rows, classic unary reaches 3,695 MiB peak memory. Chunked unary stays at 1,378 MiB, intra-job gRPC streaming at 1,495 MiB, and the RabbitMQ Streams boundary variant at 1,715 MiB. Chunking therefore reduces memory pressure consistently.

CPU is more nuanced. Chunked unary uses slightly less peak CPU than classic unary at 250k rows, but intra-job gRPC streaming and the RabbitMQ Streams boundary variant peak

higher. This is expected. Chunking replaces one large buffer with more messages, more metadata handling, more serialization boundaries, and, in the RabbitMQ Streams variant, broker stream handling and reassembly. The defensible claim is therefore that chunking improves latency and memory. CPU remains transport-dependent.

F. Failure modes and robustness

During the benchmark work, several failure modes had to be controlled before the results could be interpreted reliably. Appendix C summarizes these cases. The important distinction is that some fixes make the baseline valid, such as correcting the multi-provider merge, while others show the architectural benefit of chunking, such as avoiding single-message size limits and full-result buffering. The RQ3 robustness claim is therefore intentionally bounded: chunking removes size-dependent failure modes, reduces full-result memory pressure, and exposes partial progress, but it does not yet prove recovery from arbitrary pod crashes, broker outages, or mid-stream policy revocation inside DYNAMOS.

G. Discussion and RQ3 answer

The DYNAMOS-specific benchmark changes the interpretation of the isolated RQ1 transport study. RQ1 showed that client-streaming gRPC is the strongest raw transport in the controlled pipeline. The DYNAMOS evaluation shows that integration fit matters as much as raw transport speed. DYNAMOS is a policy-driven, dynamically composed middleware in which decoupling, generated job lifecycle, sidecar communication, and diagnosable failure behaviour are part of the design problem.

The central RQ1 hypothesis partly survives validation, but in a more precise form. Chunking is the main source of the improvement, not the name of the transport alone. RabbitMQ Streams remains useful to study as a distributed-boundary mechanism, but in the local DYNAMOS runs it adds overhead compared with ordinary chunked transfer. This is why the RQ3 conclusion ranks chunked unary highest for the current local system while keeping RabbitMQ Streams as a future distributed candidate.

For the current local DYNAMOS implementation, chunked unary is the strongest default candidate. It has the best average completion time at 100k and 250k rows, the lowest 250k memory peak, and the smallest architectural change from the original message-driven design. It also shows why the main bottleneck was not simply "gRPC versus broker", but full-result buffering across the DYNAMOS path.

Intra-job gRPC streaming remains a strong technical option. It is close to chunked unary at 250k rows and slightly faster at 50k rows. Its main weakness is architectural. Long-lived direct streams couple producer and consumer lifetime more tightly, which complicates dynamic composition, restart handling, and load balancing in Kubernetes-style deployments [3]. It is therefore best interpreted as a targeted optimisation for controlled in-pod or direct service-to-service flows.

The RabbitMQ Streams boundary variant should be interpreted carefully. It beats classic unary because it uses the common chunk-aware path, not because RabbitMQ Streams is faster than normal RabbitMQ chunks in the local benchmark. Compared with chunked unary, it is slower and uses more peak CPU and memory in this run. Its value is therefore not local latency. Its value is the stream-specific broker model: retained records, offset-based replay, non-destructive consumption, and a closer fit for distributed inter-agent transfer.

The answer to RQ3 is that the implemented chunk-aware DYNAMOS data paths improve local bulk-retrieval scalability compared with classic unary. They reduce completion time, expose earlier partial results, and lower peak memory usage for 50k, 100k, and 250k row result sets. The best current local default is chunked unary. Intra-job gRPC streaming is a close alternative for pod-local service chains. RabbitMQ Streams should be kept as a distributed-boundary candidate, but its advantage must be validated under cross-site conditions rather than inferred from the local latency result.

H. Threats to validity

The RQ3 evidence has five main limits. First, the benchmark runs on a local Kubernetes deployment, so it does not capture cross-site latency, bandwidth variation, or independent administrative domains. Second, the additional scenario checks cover two alternative request shapes, not the full space of DYNAMOS archetypes, query forms, and provider combinations. They support compatibility, but they do not replace a broader workload campaign. Third, resource usage is sampled once per second, so peak CPU and memory values can miss short spikes. Fourth, the fixed 5,000-row chunk size is justified experimentally but not optimized across all workloads. Fifth, the measured request window includes policy handling, orchestration, generated job startup, and response serialization. That is fair for comparing user-visible DYNAMOS behaviour, but it compresses the apparent transfer-only speedup because fixed control-path costs remain present in every implementation.

VI. FUTURE WORK

The local DYNAMOS benchmark is not the final distributed scalability answer. This thesis validates the local implementation first, because the basic streaming path must work before it is meaningful to test it across sites. The natural next step is Scattered Directive, which extends DYNAMOS with cross-node microservice chains for policy-aware VFL workflows and uses FABRIC to automate distributed deployment [33], [34].

In that setting, bulk transfer can become part of a larger VFL-style workflow. Parties may hold different features for overlapping entities, and a TTP or shared execution component may coordinate aligned intermediate values, labels, embeddings, model updates, or other training artefacts. The current thesis does not claim to validate a complete VFL training workload. It validates the DYNAMOS communication path that such a workload would depend on when large intermediate results have to cross generated services and agent boundaries.

FABRIC provides programmable compute and storage resources connected by high-speed dedicated links, which makes it a suitable environment for testing DYNAMOS when the TTP and data providers are separated across sites [35], [36]. A later FABRIC experiment should keep the RQ3 metrics from this section and add network throughput, broker lag or stream backlog, cross-site tail latency, failure recovery, concurrent-job scaling, and policy-change responsiveness. RabbitMQ Streams deserves special attention in that setting because its local overhead may be outweighed by retained-stream and replay semantics once agents are no longer co-located.

VII. CONCLUSION

RQ1 asks which streaming communication technologies, frameworks, and interaction patterns are suitable for large-scale and continuous data exchange in microservice-based systems such as DYNAMOS. At the level of isolated transport performance, client-streaming gRPC is the strongest raw result in the generic pipeline benchmark. Batched unary is close in the clean matrix and stronger in the backpressure scenario, while per-message unary is much slower for bulk transfer. That result establishes the value of amortising per-message overhead, whether by streaming or by batching, rather than treating streaming itself as the only useful mechanism. The brokered alternatives expose the cost and benefit of decoupling, persistence, replay, and failure handling.

RQ2 asks how streaming communication can be integrated into DYNAMOS while preserving dynamic service composition, policy-driven orchestration, and isolation. The implementation keeps the existing control path intact: requests are still evaluated and composed before execution, and generated jobs remain the execution unit. The change is in the bulk data path. Provider-side chunks must remain chunks through aggregate, algorithm, agent, and gateway stages; otherwise the system merely hides a full-buffered result behind a streaming interface.

RQ3 validates the implemented DYNAMOS variants on local bulk `dataThroughTtp` retrieval. All chunk-aware variants beat the classic unary baseline in average completion time and peak memory for 50k, 100k, and 250k row result sets. They also expose earlier partial results, which classic unary cannot do because its first result is its final result. Focused checks on `computeToData` and multi-provider `dataThroughTtp` show that the implementation is not confined to one request shape. The strongest local recommendation is chunked unary. It gives the best 100k and 250k averages, the lowest 250k memory peak, and the smallest architectural shift from the original message-driven design.

The more general conclusion is boundary-specific. Intra-job gRPC streaming is useful where a direct stream between generated services is acceptable. RabbitMQ Streams is not the source of the local speedup by itself, because that speedup mainly comes from chunk preservation. Its stronger case is distributed inter-agent transfer, where retained records, replay, and independent consumers may matter more than local latency. The Scattered Directive and FABRIC context is

therefore future work: it moves DYNAMOS toward cross-node VFL workflows, where network separation, broker placement, policy changes, and multi-party scaling can dominate local transfer costs.

The main research question asks how streaming communication can be integrated into DYNAMOS without weakening its policy-driven and adaptive character. The answer is to treat streaming as a data-path change after policy approval, not as a replacement for policy enforcement or orchestration. Compliance is preserved by construction in the evaluated implementation because the policy decision point, generated job composition, provider selection, and data-steward isolation remain unchanged. What changes is how approved bulk results are transferred once a job is already allowed to run. A stronger compliance claim, such as reacting to policy revocation during an active stream, remains future work.

REFERENCES

- [1] J. Stutterheim, "Dynamos: Dynamically adaptive microservice-based os: A middleware for data exchange systems," Master's thesis, University of Amsterdam, 2023.
- [2] J. Stutterheim, A. Mifsud, and A. Opreescu, "Dynamos: Dynamic microservice composition for data-exchange systems, lessons learned," in *Proceedings of the 2024 IEEE 21st International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 2024, pp. 8–15.
- [3] C. Starck and J. Ghofrani, "Improving load balancing of long-lived streaming rpcs for grpc-enabled inter-service communication," in *Proceedings of the 15th Central European Workshop on Services and their Composition (ZEUS 2023)*, ser. CEUR Workshop Proceedings, S. Böhm and D. Lübke, Eds., vol. 3386, Hannover, Germany, 2023, pp. 45–51. [Online]. Available: <https://ceur-ws.org/Vol-3386/>
- [4] A. Sodankoor, A. Shenoy, B. M. Moger, M. Gowda, P. K. Auradkar, and S. Kalambur, "Benchmarking grpc protocol on various virtualization technologies," in *ICT4SD 2025*, ser. Lecture Notes in Networks and Systems. Springer Nature Switzerland AG, 2026, vol. 1654, pp. 316–329.
- [5] A. G. Ibrahim, R. P. Lopes, J. Rufino, and P. Leitão, "On the impact of message brokers implementations in the choreography of microservices," in *Open Lecture Series 2A (OL2A 2025)*, ser. Communications in Computer and Information Science (CCIS). Springer Nature Switzerland AG, 2026.
- [6] J. Nuiikka, "Comparison of cloud native messaging technologies," Master's thesis, Tampere University, Faculty of Information Technology and Communication Sciences, May 2021. [Online]. Available: <https://urn.fi/URN:NBN:fi:tuni-202104223314>
- [7] G. Coviello, K. Rao, C. G. D. Vita, G. Mellone, and S. Chakradhar, "Datax: Flexible and efficient communication in microservices-based stream analytics pipelines," in *2022 IEEE Intl Conf on Dependable, Autonomic and Secure Computing (DASC) / PiCom / CBDCom / CyberSciTech*, 2022.
- [8] J. Karimov, T. Rabl, A. Katsifodimos, R. Samarev, H. Heiskanen, and V. Markl, "Benchmarking distributed stream data processing systems," in *Proceedings of the IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 2018, pp. 1519–1530.
- [9] S. Henning and W. Hasselbring, "How to measure scalability of distributed stream processing engines?" in *Companion of the ACM/SPEC International Conference on Performance Engineering (ICPE '21)*. ACM, 2021, pp. 85–88.
- [10] S. Ris, J. Araujo, and D. Beserra, "A systemic mapping of methods and tools for performance analysis of data streaming with containerized microservices architecture," in *2023 18th Iberian Conference on Information Systems and Technologies (CISTI)*, 2023, pp. 1–6.
- [11] B. Çiftçi and B. Çiloğlul, "A review of comparative studies on performance evaluation of communication mechanisms for microservices," in *Computational Science and Its Applications – ICCSA 2025*, ser. Lecture Notes in Computer Science. Springer Nature Switzerland AG, 2025, vol. 15650, pp. 98–113.
- [12] J. Figueira and C. Coutinho, "Developing self-adaptive microservices," *Procedia Computer Science*, vol. 232, pp. 264–273, 2024.
- [13] S. Neumaier, G. Havur, and T. Pellegrini, "Towards an architecture for policy-aware decentral dataset exchange," in *Proceedings of the Fifteenth International Conference on Advances in Semantic Processing (SEMAPRO 2021)*. Barcelona, Spain: IARIA, Oct. 2021. [Online]. Available: https://www.researchgate.net/publication/358816711-Towards_an_Architecture_for_Policy-Aware_Decentral_Dataset_Exchange
- [14] G. Thiagarajan, V. Bist, and P. Nayak, "Strengthening grpc security in microservices: A proxy-based approach for mtls, jwt, and rbac enforcement," *International Journal of Computer Applications*, vol. 187, no. 28, Aug. 2025.
- [15] J. A. Rasheed and S. Saradha, "Reactive microservices architecture using a framework of fault tolerance mechanisms," in *Proceedings of the Second International Conference on Electronics and Sustainable Communication Systems (ICESC)*. IEEE, 2021.
- [16] B. Guntupalli and S. Vamshi Ch, "Designing microservices that handle high-volume data loads," *International Journal of AI, Big Data, Computational and Management Studies*, vol. 4, no. 4, pp. 76–87, 2023.
- [17] Y. Gan and C. Delimitrou, "The architectural implications of cloud microservices," *IEEE Computer Architecture Letters*, vol. 17, no. 2, pp. 155–158, 2018.
- [18] M. Roohitavaf, K. Ren, G. Zhang, and S. Ben-Romdhane, "Logplayer: Fault-tolerant exactly-once delivery using grpc asynchronous streaming," 2019, arXiv preprint.
- [19] Ritu, S. Arora, A. Bhardwaj, A. Kukkar, and S. Kaur, "A comparative analysis of communication efficiency: Rest vs. grpc in microservice-based ecosystems," in *Proceedings of the 2024 International Conference on Emerging Innovations and Advanced Computing (INNOCOMP)*. IEEE, 2024.
- [20] V. Lähdevänoja, "Communication methods and protocols between microservices on a public cloud platform," Master's thesis, Aalto University, School of Science, Espoo, Finland, May 2021. [Online]. Available: <https://urn.fi/URN:NBN:fi:aalto-202106207501>
- [21] gRPC Authors. (2024) Core concepts, architecture and lifecycle. Last modified 2024-11-12. [Online]. Available: <https://grpc.io/docs/what-is-grpc/core-concepts/>
- [22] Apache Software Foundation. (2026) Apache kafka. [Online]. Available: <https://kafka.apache.org/>
- [23] P. Dobbelaere and K. Sheykh Esmaili, "Kafka versus rabbitmq," 2017, arXiv preprint.
- [24] M. Mohammad, "Analysis of design patterns and benchmark practices in apache kafka event-streaming systems," 2025, arXiv preprint.
- [25] T. Sharvari and K. Sowmya Nag, "A study on modern messaging systems: Kafka, rabbitmq and nats streaming," 2019, arXiv preprint.
- [26] RabbitMQ Team. (2026) Streams and superstreams (partitioned streams). [Online]. Available: <https://www.rabbitmq.com/docs/streams>
- [27] The NATS Authors. (2026) Overview. [Online]. Available: <https://docs.nats.io/nats-concepts/overview>
- [28] The NATS Authors. (2026) Request-reply. [Online]. Available: <https://docs.nats.io/nats-concepts/core-nats/requeple>
- [29] The NATS Authors. (2026) Jetstream. [Online]. Available: <https://docs.nats.io/nats-concepts/jetstream>
- [30] A. Vogel, S. Henning, E. Perez-Wohlfeil, O. Ertl, and R. Rabiser, "A comprehensive benchmarking analysis of fault recovery in stream processing frameworks," in *Proceedings of the 18th ACM International Conference on Distributed and Event-Based Systems (DEBS '24)*. ACM, 2024, pp. 171–182.
- [31] DYNAMOS Contributors, "DYNAMOS source repository, main branch, commit a2d9531," <https://github.com/neburpoots/DYNAMOS/tree/a2d953166c369260e6014f41bd988ee3ae45592d>, 2026, accessed: 2026-06-09.
- [32] DYNAMOS Contributors, "DYNAMOS streaming implementations prototype, feature/streaming-implementations branch, commit 923b018," <https://github.com/neburpoots/DYNAMOS/tree/923b018e622e30ed6141a3e9c925417ce9150e5d>, 2026, accessed: 2026-05-25.
- [33] J. Jongejans, A. Koufakis, and A. Opreescu, "Vertical federated learning on scattered directive: Enforcing policies on VFL workflows," in *Proceedings of the 2025 IEEE International Conference on Big Data (BigData)*, 2025, pp. 267–274.
- [34] DYNAMOS-UVA, "Scattered directive source repository," <https://github.com/DYNAMOS-UVA/Scattered-Directive>, 2026, accessed: 2026-05-25.
- [35] I. Baldin, A. Nikolich, J. Griffioen, I. I. S. Monga, K.-C. Wang, T. Lehman, P. Ruth, and E. Deelman, "FABRIC: A national-scale programmable experimental network infrastructure," *IEEE Internet Computing*, vol. 23, no. 6, pp. 38–47, 2019.
- [36] FABRIC Testbed, "About FABRIC," <https://portal.fabric-testbed.net/about/about-fabric>, 2026, accessed: 2026-05-25.

APPENDIX A BENCHMARK PRESET DEFINITIONS

This appendix documents the complete preset definitions used in the benchmark evidence reported in the thesis. Each preset specifies the workload family, payload configuration, concurrency level, target rate where applicable, failure injection parameters, and resource overlay tier. The four reported presets are: *synthetic-clean* for bulk transfer, *synthetic-continuous* for steady-rate pacing, *synthetic-backpressure* for slow-consumer overload, and *synthetic-recovery* for forced-restart resilience. The harness also contains a CSV replay mode, but those results are not used in the current thesis argument.

Table XI
BENCHMARK PRESETS USED IN THE CURRENT TRANSPORT STUDY.

Preset	Key parameters	Purpose
<i>synthetic-clean</i>	Synthetic payloads from 256 B to 16 KiB; concurrency 1–16; five repetitions; no injected failure	Baseline bulk-transfer throughput, latency, and correctness matrix
<i>synthetic-continuous</i>	1 KiB payloads, 1,000 msg/s target, concurrency 4 and 8, five repetitions	Steady-rate pacing and jitter comparison
<i>synthetic-backpressure</i>	1 KiB payloads, 4,000 msg/s target, concurrency 8, 2 ms sink delay, five repetitions	Slow-consumer overload and queueing behaviour
<i>synthetic-recovery</i>	4 KiB payloads, 2,000 msg/s target, concurrency 8, transformer restart after 3 s, 120 retries at 250 ms backoff, five repetitions	Restart recovery, replay, and continuity analysis

APPENDIX B FULL RUN MATRIX

This appendix summarizes the canonical run scopes used in the current thesis. The body of the thesis presents the most decision-relevant subsets, while the benchmark result directory stores the corresponding per-run JSON, NDJSON, and CSV exports generated by the execution scripts. The chapter is based on the single-node six-transport *synthetic-clean* matrix and the matching single-node scenario suite, each with five repetitions. Clustered broker runs are kept as supporting variability context rather than as the primary thesis evidence.

Table XII
EXECUTION SCOPE OF THE CURRENT BENCHMARK CORPUS.

Scope	Transports	Variable dimensions	Reps	Role in thesis
Single-node clean matrix	Client-streaming, batched unary, RabbitMQ, NATS, Kafka	Payload 256 B–16 KiB, concurrency 1–16	5	Full bulk-transfer baseline (Section III)
Single-node continuous	All six	Concurrency 4 and 8, 1 KiB, 1,000 msg/s	5	Pacing and jitter evidence
Single-node backpressure	All six	1 KiB, concurrency 8, 4,000 msg/s, 2 ms sink delay	5	Overload and queueing evidence
Single-node recovery	All six	4 KiB, concurrency 8, transformer restart	5	Recovery and correctness evidence
Clustered clean (3-broker)	RabbitMQ, NATS, Kafka	4 KiB, concurrency 8	3	Supporting broker variability context
Clustered continuous (3-broker)	RabbitMQ, NATS, Kafka	1 KiB, concurrency 8, 1,000 msg/s	3	Clustered pacing under broker replication
Clustered backpressure (3-broker)	RabbitMQ, NATS, Kafka	1 KiB, concurrency 8, 4,000 msg/s, 2 ms delay	3	Clustered overload behaviour
Clustered recovery (3-broker)	RabbitMQ, NATS, Kafka	4 KiB, concurrency 8, transformer restart	3	Clustered recovery correctness and variability

APPENDIX C DYNAMOS FAILURE-MODE NOTES

APPENDIX D DETAILED CLEAN-MATRIX RESULTS

This appendix records the completed six-transport *synthetic-clean* matrix at the payload and concurrency level. Values are arithmetic means over five repetitions. The result directory stores the per-run JSON and CSV files for the individual repetitions.

APPENDIX E RESOURCE UTILISATION DETAILS

This appendix summarizes supporting resource metrics for diagnostic Kubernetes scenario runs where CPU and memory data are most informative. For brokered modes, the broker is reported as a separate role because its buffering layer is a core part of the transport cost. These tables are not used for the primary RQ1 ranking, which is based on the completed five-repetition throughput, latency, and correctness corpus. The six-transport scenario resource exports remain in the benchmark result directory; the tables below are retained as role-level context where the sampling scope is comparable.

Table XIII
FAILURE MODES OBSERVED OR ACCOUNTED FOR DURING THE DYNAMOS BENCHMARK WORK.

Failure mode	Affected path	Observation	Treatment	Robustness claim
Default gRPC message ceiling	Original classic unary	Original DYNAMOS documentation reports a practical limit around 4 MB, or roughly 30k rows in that setup	Increased benchmark message limits for the baseline so the comparison measures buffering cost rather than an artificial ceiling	Chunking removes the dependency on one very large response message
Full-result buffering	Classic unary and early bulk prototypes	Large results were materialized repeatedly before the client saw progress	Preserved 5k-row chunks through SQL, aggregate, algorithm, agent, and gateway stages	Supported by lower memory peaks and earlier partial results
Stale generated jobs	Repeated classic unary runs	Later repetitions could return fast 502 or empty-provider responses when old generated jobs remained	Cleaned generated <code>surf</code> , <code>uva</code> , and <code>vu</code> jobs outside the measured request window	Not a streaming advantage; handled for benchmark fairness
Multi-provider merge error	Classic baseline multi-provider	The first provider could be appended twice during aggregation	Applied a minimal aggregation correction before reporting multi-provider baseline timings	Baseline correctness fix, not a streaming feature
Timeout and empty-provider reporting	Early large-result runs	Some failures appeared as missing final results or empty provider bodies	Made timeout/error outcomes visible in benchmark output and separated clean errors from valid completions	Improves diagnosability; not a full crash-recovery proof

Table XIV
SYNTHETIC BULK-TRANSFER THROUGHPUT BY MESSAGE SIZE, AVERAGED OVER FIVE REPETITIONS AND CONCURRENCY LEVELS 1, 4, 8, AND 16.

Msg size	Stream msg/s	Batched unary	Unary msg/s	RabbitMQ msg/s	NATS msg/s	Kafka msg/s
256 B	98,403	91,177	3,688	23,498	24,136	16,997
1 KiB	70,865	65,806	3,352	18,165	17,564	14,955
4 KiB	29,569	36,564	3,036	9,916	11,214	9,543
16 KiB	13,126	11,992	2,244	4,023	5,187	3,376

Table XV
DETAILED SYNTHETIC-CLEAN MESSAGE THROUGHPUT (MSG/S) BY PAYLOAD SIZE AND CONCURRENCY, AVERAGED OVER FIVE REPETITIONS.

Msg size	Concur.	Stream	Batched	Unary	RabbitMQ	NATS	Kafka
256 B	1	94,624	50,297	892	21,858	20,013	19,310
	4	95,861	78,821	2,904	26,553	23,538	16,505
	8	119,841	116,198	4,566	24,890	25,813	16,577
	16	83,286	119,391	6,391	20,691	27,180	15,598
1 KiB	1	73,865	38,530	1,001	16,264	15,920	18,370
	4	76,193	77,699	2,583	18,621	18,681	13,710
	8	72,650	76,337	4,173	19,136	18,381	15,709
	16	60,753	70,658	5,651	18,640	17,274	12,032
4 KiB	1	28,011	25,658	986	8,752	10,493	9,870
	4	32,767	37,108	2,450	10,738	12,656	10,733
	8	29,574	42,256	3,758	9,756	12,980	10,034
	16	27,924	41,234	4,948	10,418	8,730	7,537
16 KiB	1	10,962	9,405	823	3,140	4,796	3,127
	4	14,263	12,795	2,118	4,739	5,888	3,521
	8	13,775	13,249	2,276	4,343	5,472	3,694
	16	13,503	12,518	3,758	3,869	4,594	3,163

Table XVI
DETAILED SYNTHETIC-CLEAN DATA THROUGHPUT (MiB/S) BY PAYLOAD SIZE AND CONCURRENCY, AVERAGED OVER FIVE REPETITIONS.

Msg size	Concur.	Stream	Batched	Unary	RabbitMQ	NATS	Kafka
256 B	1	23.10	12.28	0.22	5.34	4.89	4.71
	4	23.40	19.24	0.71	6.48	5.75	4.03
	8	29.26	28.37	1.11	6.08	6.30	4.05
	16	20.33	29.15	1.56	5.05	6.64	3.81
1 KiB	1	72.13	37.63	0.98	15.88	15.55	17.94
	4	74.41	75.88	2.52	18.18	18.24	13.39
	8	70.95	74.55	4.08	18.69	17.95	15.34
	16	59.33	69.00	5.52	18.20	16.87	11.75
4 KiB	1	109.42	100.23	3.85	34.19	40.99	38.55
	4	127.99	144.95	9.57	41.95	49.44	41.93
	8	115.52	165.06	14.68	38.11	50.70	39.20
	16	109.08	161.07	19.33	40.69	34.10	29.44
16 KiB	1	171.29	146.95	12.86	49.07	74.93	48.86
	4	222.85	199.92	33.09	74.04	92.00	55.02
	8	215.24	207.02	35.56	67.86	85.49	57.72
	16	210.98	195.59	58.72	60.46	71.78	49.43

Table XVII
DETAILED SYNTHETIC-CLEAN p_{95} LATENCY (MS) BY PAYLOAD SIZE AND CONCURRENCY, AVERAGED OVER FIVE REPETITIONS.

Msg size	Concur.	Stream	Batched	Unary	RabbitMQ	NATS	Kafka
256 B	1	21.8	1.3	0.5	273.8	507.9	199.6
	4	70.0	9.4	0.8	277.2	448.6	569.3
	8	73.8	26.7	0.8	373.6	432.5	583.7
	16	87.6	40.4	1.2	449.6	434.4	1,179.2
1 KiB	1	39.5	2.4	0.5	379.3	677.8	177.6
	4	79.0	5.9	0.7	426.5	614.6	652.3
	8	91.0	38.4	1.0	432.6	685.0	717.5
	16	129.5	59.9	1.4	479.4	984.9	1,644.8
4 KiB	1	33.6	4.6	0.6	507.1	1,078.6	167.8
	4	109.2	45.2	0.7	713.0	929.8	824.8
	8	171.0	64.6	1.2	1,241.1	1,195.4	1,128.2
	16	187.4	81.9	2.3	809.7	1,943.4	2,235.7
16 KiB	1	49.1	39.2	0.6	1,856.6	2,794.2	100.8
	4	103.5	68.2	1.4	1,511.0	3,876.9	2,065.0
	8	301.2	93.0	2.4	2,136.8	3,850.6	2,030.4
	16	191.6	245.0	6.0	2,634.4	3,836.9	9,163.6

Table XVIII
CONTINUOUS SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Concur.	Producer	Transformer	Sink	Broker
Client-streaming	4	23.40 / 23.40	8.99 / 24.20	5.61 / 14.90	–
Client-streaming	8	–	9.31 / 25.60	5.64 / 15.50	–
Unary	4	35.70 / 35.70	17.88 / 39.10	11.12 / 24.10	–
Unary	8	–	15.54 / 33.70	9.72 / 20.90	–
RabbitMQ Streams	4	7.92 / 8.30	9.87 / 15.60	6.91 / 10.90	29.88 / 43.30
RabbitMQ Streams	8	7.88 / 8.20	10.46 / 15.80	7.50 / 11.70	33.00 / 46.50
NATS JetStream	4	–	12.62 / 24.90	5.60 / 15.40	18.37 / 40.90
NATS JetStream	8	–	16.23 / 23.70	6.45 / 14.20	22.74 / 38.50
Kafka	4	–	4.68 / 5.70	0.69 / 1.90	51.52 / 52.90
Kafka	8	–	8.58 / 8.80	1.00 / 2.20	41.13 / 58.80

Table XIX
RECOVERY SCENARIO CPU USAGE BY ROLE (AVG./PEAK %).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	31.60 / 31.60	12.64 / 31.60	3.89 / 17.50	–
Unary	4.60 / 6.00	2.45 / 9.10	2.56 / 6.10	–
RabbitMQ Streams	8.84 / 10.20	13.38 / 17.10	7.51 / 13.90	31.76 / 49.90
NATS JetStream	0.30 / 0.30	17.29 / 25.20	7.28 / 16.20	20.99 / 43.50
Kafka	0.40 / 0.40	7.63 / 8.10	1.44 / 2.50	46.77 / 75.40

Table XX
RECOVERY SCENARIO MEMORY USAGE BY ROLE (AVG./PEAK MiB).

Mode	Producer	Transformer	Sink	Broker
Client-streaming	7.00 / 7.00	3.40 / 7.00	2.33 / 7.00	–
Unary	7.00 / 7.00	3.08 / 7.00	4.53 / 7.00	–
RabbitMQ Streams	10.00 / 10.00	25.72 / 31.00	5.78 / 8.00	490.91 / 511.00
NATS JetStream	8.00 / 8.00	14.00 / 14.00	9.75 / 15.00	306.06 / 537.00
Kafka	10.00 / 10.00	8.00 / 8.00	6.53 / 9.00	449.71 / 478.00